

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	R.Uday Kumar Reddy	Reg. No.:	192424190
Section / Batch:	Slot-C	Date:	22/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Q27. Knapsack Problem vs Container Loading (DP vs Greedy)

Given weights and capacity, solve using 0/1 Knapsack (DP) and Container Loading (Greedy). Compare selected items and utilization efficiency. Analyze why greedy fails for discrete optimization. Evaluate computational complexity. Discuss real-world logistics applications. Generate insights on solution strategies. Instructions: Solve the assigned problem, show the algorithm/steps clearly, analyze correctness, time and space complexity, and provide the requested comparison or insights.

Code of Conduct

I R. Uday Kumar Reddy (192424190)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: R. Uday_____

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q27. Knapsack Problem vs Container Loading (DP vs Greedy)

Given weights and capacity, solve using 0/1 Knapsack (DP) and Container Loading (Greedy). Compare selected items and utilization efficiency. Analyze why greedy fails for discrete optimization. Evaluate computational complexity. Discuss real-world logistics applications. Generate insights on solution strategies. Instructions: Solve the assigned problem, show the algorithm/steps clearly, analyse correctness, time and space complexity, and provide the requested comparison or insights.

1. Problem Statement

Given a set of items with weights and a container capacity, solve the problem using:

- 1. 0/1 Knapsack using Dynamic Programming (DP)**
- 2. Container Loading using Greedy Method**

Then compare:

- Selected items
- Total weight
- Capacity utilization
- Correctness
- Time and space complexity
- Why greedy can fail for discrete optimization
- Real-world logistics applications

2. Example

Assume the container capacity is:

Capacity = 10 kg

Item Weight

- A 6 kg
- B 4 kg
- C 5 kg
- D 3 kg
- E 2 kg

For simplicity, assume each item has a value equal to its weight when comparing pure container utilization.

3. 0/1 Knapsack Using Dynamic Programming

Idea

In 0/1 Knapsack, each item can be either:

1 → Selected

0 → Not selected

We cannot take a fraction of an item.

The DP recurrence is:

$DP[i][w] =$

$\{ DP[i-1][w] \text{ if } \text{weight}[i] > w$

$\{ \max(DP[i-1][w], DP[i-1][w - \text{weight}[i]] + \text{value}[i]) \text{ otherwise}$

Since value = weight here, we want to maximize the total weight without exceeding 10 kg.

DP Table

Items / Capacity	0	1	2	3	4	5	6	7	8	9	10
None	0	0	0	0	0	0	0	0	0	0	0
A=6	0	0	0	0	0	0	6	6	6	6	6
B=4	0	0	0	0	4	4	6	6	6	6	10
C=5	0	0	0	0	4	5	6	6	6	9	10
D=3	0	0	0	3	4	5	6	7	8	9	10
E=2	0	0	2	3	4	5	6	7	8	9	10

The maximum achievable weight is:

10 kg

Selected items

The DP solution can select:

$A+B=6+4=10$

or

$A+D+E=6+3+2=11$

which is not allowed.

Another valid optimal combination is:

$$B+C+D=4+5+3=12$$

not allowed.

Therefore:

A+B=10 kg is an optimal selection.

Utilization

$$\begin{aligned} Utilization &= \frac{Selected\ Weight}{Capacity} \times 100 \\ &= \frac{10}{10} \times 100 \\ &= \boxed{100\%} \end{aligned}$$

4. Container Loading Using Greedy

A common greedy strategy is:

Select the largest item that fits into the remaining capacity.

Sort the items in descending order:

6, 5, 4, 3, 2

Capacity = 10 kg.

Step 1

Select **6 kg**.

Remaining:

$$10-6=4$$

Step 2

5 kg does not fit.

Select **4 kg**.

Remaining:

$$4-4=0$$

Selected:

$$6+4=10$$

So this particular example gives:

10 kg and utilization:100%

5. Example Where Greedy Fails

To understand why greedy is not always optimal, consider:

Capacity = 10 kg

Item	Weight	Value
A	6	12
B	5	10
C	4	9
D	4	8

Suppose greedy chooses the largest weight first.

Greedy

Select A:

6 kg, value=12

Remaining capacity:

$$10-6=4$$

Select C:

$$6+4=10$$

Total value:

$$12+9=21$$

DP

DP examines all combinations.

B + C:

$$5+4=9$$

Value:

$$10+9=19$$

A + C:

$$6+4=10$$

Value:

$$12+9=21$$

So in this example greedy happens to work.

Let's modify the values:

Item	Weight	Value
A	6	12
B	5	11
C	5	11

Greedy selects A:

6 kg,12

Remaining capacity = 4, so no other item fits.

Greedy value:

12

But DP selects B + C:

$$5+5=10$$

Value:

$$11+11=22$$

Therefore:

$$DP=22 > Greedy=12$$

This clearly demonstrates greedy failure.

6. Why Does Greedy Fail?

Greedy makes a decision based on the **current best choice**.

It does not normally reconsider previous decisions.

For example:

Capacity = 10

Greedy:

Choose 6

Remaining = 4

Nothing useful fits

Total value = 12

But the optimal solution is:

Choose 5

Choose 5

Total weight = 10

Total value = 22

The problem is that the locally best choice does not necessarily produce the globally best solution.

Main reason

0/1 Knapsack has:

- Discrete decisions
- Interdependent choices
- No guarantee that the greedy-choice property holds

Therefore, Dynamic Programming is required to guarantee the optimal solution.

7. Algorithm Steps

A. 0/1 Knapsack — DP

Steps

1. Take n items and capacity W .
2. Create a DP table of size $(n+1) \times (W+1)$.
3. Initialize the first row and first column to 0.
4. For every item:
 - If its weight is greater than the current capacity, don't select it.
 - Otherwise, compare:
 - Excluding the item
 - Including the item

5. Store the maximum value.
6. Trace backward through the table to identify selected items.

Pseudocode

Knapsack(items, W):

for w = 0 to W:

DP[0][w] = 0

for i = 1 to n:

for w = 0 to W:

if weight[i] ≤ w:

DP[i][w] =

max(DP[i-1][w],

value[i] + DP[i-1][w-weight[i]])

else:

DP[i][w] = DP[i-1][w]

return DP[n][W]

8. Container Loading — Greedy

Steps

1. Sort items according to the selected greedy criterion.
2. Start with the full container capacity.
3. Select the next item if it fits.
4. Reduce the remaining capacity.
5. Continue until no more items can be loaded.

Pseudocode

ContainerLoading(items, Capacity):

sort items in descending order of weight

remaining = Capacity

selected = []

for each item:

if item.weight ≤ remaining:

```

selected.append(item)

remaining = remaining - item.weight

return selected

```

9. Correctness Analysis

0/1 Knapsack

DP considers **both possibilities** for every item:

1. Include the item
2. Exclude the item

Therefore, it explores all relevant combinations through subproblems and guarantees the optimal solution.

Greedy Container Loading

Greedy only considers the immediate best choice.

It does **not** consider every combination.

Therefore:

However, greedy can be very useful when:

- Approximate solutions are acceptable
- Speed is more important than optimality
- The problem has a proven greedy-choice property

10. Complexity Comparison

Feature	0/1 Knapsack — DP	Container Loading — Greedy
Approach	Dynamic Programming	Greedy
Optimality	Guaranteed	Not guaranteed generally
Time Complexity	$O(nW)$	$O(n \log n)$ due to sorting
Space Complexity	$O(nW)$	$O(n)$
Decision	Global optimization	Local optimization
Suitable for	Discrete optimization	Fast packing/loading
Computation	Higher	Lower

Here:

- n = number of items , W = container capacity

11. Utilization Efficiency

- Utilization is:

$$Utilization = \frac{Loaded\ Weight}{Container\ Capacity} \times 100$$

For example:

DP

Capacity = 10 kg

Loaded = 10 kg

$10/10 \times 100 = 100\%$

Greedy failure example

Suppose greedy loads only 6 kg:

$6/10 \times 100 = 60\%$

Therefore:

Method	Loaded Weight	Capacity	Utilization
DP	10 kg	10 kg	100%
Greedy	6 kg	10 kg	60%

DP provides better utilization in this example.

12. Real-World Logistics Applications

1. Truck Loading

Companies need to decide which packages should be loaded into a truck while respecting:

- Weight limits
- Package value
- Space limitations
- Delivery priorities

2. Shipping Containers

A shipping company can select products to maximize:

- Container utilization

- Revenue
- Product priority

3. Airline Cargo

Cargo selection can consider:

- Aircraft weight limits
- Cargo value
- Destination
- Space requirements

4. Warehouse Management

Warehouses can optimize which products should be dispatched when truck capacity is limited.

5. Delivery Services

Delivery vehicles can select packages based on:

- Weight
- Profit
- Destination
- Vehicle capacity

13. Key Insights

Insight 1

DP is better when finding the exact optimal combination is important.

Insight 2

Greedy is faster and simpler, making it attractive for large-scale logistics.

Insight 3

Greedy may produce a poor solution because selecting one large/high-value item can prevent several smaller items from being selected.

Insight 4

For large real-world logistics problems, hybrid methods can be used:

Greedy → Generate an initial solution

↓

Local Search / DP / Optimization



Improved Solution

Insight 5

The choice depends on the objective:

Need guaranteed optimal solution?



DP

Need very fast approximate solution?



Greedy

14. Final Comparison

Parameter	0/1 Knapsack	Container Loading
Technique	Dynamic Programming	Greedy
Item selection	0 or 1	Select if it fits
Global optimization	Yes	No
Optimal solution	Guaranteed	Not guaranteed
Time	$O(nW)$	$O(n \log n)$
Space	$O(nW)$	$O(n)$
Utilization	Usually optimal for the defined objective	May be lower
Implementation	More complex	Simple
Large-scale use	Computationally expensive	Fast and practical

Conclusion

The **0/1 Knapsack problem using Dynamic Programming** provides an optimal solution because it evaluates the possible inclusion and exclusion of items and uses previously solved subproblems. In contrast, **Greedy Container Loading** is faster and easier to implement because it makes decisions based on the current best choice. However, greedy can fail for discrete optimization because a locally optimal choice may prevent a better combination later.